Lecture 4. Least Squares: QR Decomposition

Gram–Schmidt, Householder, Givens, conditioning of least squares, underdetermined problems

Contents

4.1 QR decomposition and least squares **2**

4.2 Gram–Schmidt **2**

4.3 Householder reflections **3**

4.4 Givens rotations **4**

4.5 Conditioning of least squares **4**

4.6 Underdetermined least squares **5**

4.1 QR decomposition and least squares

Cholesky on the normal equations is one way to do least squares; the QR decomposition is another, and is what R's `lm` uses. Let $X \in \mathbb{R}^{n\times p}$ have full column rank ($n \geq p$).

Definition (QR decomposition)

- **Full QR**: $X = QR$ with $Q \in \mathbb{R}^{n\times n}$ orthogonal and $R \in \mathbb{R}^{n\times p}$ upper triangular (its lower $(n-p)\times p$ block is 0). The first p columns of Q are an orthonormal basis of the column space $\mathcal{R}(X)$, the last $n-p$ of $\mathcal{N}(X^\top) = \mathcal{R}(X)^\perp$.
- **Reduced QR**: $X = Q_1 R_1$ with $Q_1 \in \mathbb{R}^{n\times p}$, $Q_1^\top Q_1 = I_p$ (but $Q_1 Q_1^\top \neq I_n$), and $R_1 \in \mathbb{R}^{p\times p}$ upper triangular with positive diagonal.

Since $X^\top X = R^\top Q^\top Q R = R_1^\top R_1$, a QR decomposition of X gives the Cholesky factor of the Gram matrix for free.

Least squares via QR. The normal equations $X^\top X\beta = X^\top y$ become $R_1^\top R_1\beta = R_1^\top Q_1^\top y$, and since R_1 is invertible,

$$R_1\beta = Q_1^\top y,$$

a single back substitution. (Equivalently, $\|y - X\beta\|^2 = \|Q^\top y - R\beta\|^2 = \|Q_1^\top y - R_1\beta\|^2 + \|Q_2^\top y\|^2$, minimized when the first term vanishes, and the residual sum of squares is $\|Q_2^\top y\|^2$.) This is more accurate than solving the normal equations because

$$\kappa(X^\top X) = \kappa(X)^2,$$

since the singular values of $X^\top X$ are the squares of those of X. Standard errors need $(X^\top X)^{-1} = R_1^{-1}R_1^{-\top}$, again via triangular solves.

4.2 Gram–Schmidt

Every full-column-rank X has a reduced QR decomposition: it is the Gram–Schmidt orthonormalization of the columns $x_1, \ldots, x_p$.

(1) $q_1 = x_1/\|x_1\|_2$.

(2) For $k = 2, \ldots, p$: $v_k = x_k - \sum_{j<k}\langle q_j, x_k\rangle q_j$ and $q_k = v_k/\|v_k\|_2$.

The q_k are orthonormal with $\operatorname{span}\{x_1, \ldots, x_k\} = \operatorname{span}\{q_1, \ldots, q_k\}$ for each k. With $r_{jk} = \langle q_j, x_k\rangle$ $(j < k)$, $r_{kk} = \|v_k\|_2$ and $r_{jk} = 0$ $(j > k)$, step (2) reads $x_k = \sum_{j\leq k} r_{jk} q_j$, i.e. $X = Q_1 R_1$.

Classical vs modified. Classical Gram–Schmidt (CGS) computes all $\langle q_j, x_k\rangle$ with the original x_k. When columns are nearly collinear, rounding errors make the computed q's far from orthogonal. **Modified** Gram–Schmidt (MGS) subtracts the projections one at a time, projecting the *current* partially orthogonalized vector:

$$v_k^{(1)} = x_k, \qquad v_k^{(j+1)} = v_k^{(j)} - \langle q_j, v_k^{(j)}\rangle q_j, \quad j = 1, \ldots, k-1, \qquad q_k = v_k^{(k)}/\|v_k^{(k)}\|.$$

In exact arithmetic the two agree; in floating point MGS loses much less orthogonality, though it is not fully immune. Both cost $\sum_k 4n(k-1) \approx 2np^2$ flops and give only the reduced QR.

Example 4.1 (Where CGS breaks)

Let $x_1 = (1, \epsilon, 0, 0)^\top$, $x_2 = (1, 0, \epsilon, 0)^\top$, $x_3 = (1, 0, 0, \epsilon)^\top$ with ϵ^2 below the unit roundoff, so that $[1 + \epsilon^2] = 1$. Then $r_{11} = [\sqrt{1 + \epsilon^2}] = 1$ and $q_1 = x_1$; the computed r_{11} has dropped the ϵ^2 that should have entered later inner products.

- $k = 2$: $r_{12} = q_1^\top x_2 = 1$, $v_2 = (0, -\epsilon, \epsilon, 0)^\top$, $q_2 = (0, -1, 1, 0)^\top/\sqrt{2}$.
- $k = 3$, CGS: $r_{13} = q_1^\top x_3 = 1$ and $r_{23} = q_2^\top x_3 = 0$ (both with the *original* x_3), so $v_3 = (0, -\epsilon, 0, \epsilon)^\top$ and $q_3 = (0, -1, 0, 1)^\top/\sqrt{2}$. Now $q_2^\top q_3 = \frac{1}{2}$: orthogonality is completely lost.
- $k = 3$, MGS: after removing q_1, $v_3^{(2)} = (0, -\epsilon, 0, \epsilon)^\top$ and $q_2^\top v_3^{(2)} = \epsilon/\sqrt{2} \neq 0$, so the second projection is removed: $v_3 = (0, -\frac{\epsilon}{2}, -\frac{\epsilon}{2}, \epsilon)^\top$, $q_3 = (0, -1, -1, 2)^\top/\sqrt{6} \perp q_2$.

The true $q_2^\top x_3$ is a tiny nonzero number that CGS rounds away; MGS recomputes it from the partially orthogonalized vector, where it is no longer negligible relative to the vector's size. MGS fails in turn when two columns are almost parallel, e.g. $x_1 = (0.7, 0.7 + \epsilon_{\text{mach}})^\top$ and $x_2 = (1/\sqrt{2}, 1/\sqrt{2})^\top$: then $v_2 = x_2 - \langle q_1, x_2\rangle q_1$ is a difference of nearly equal vectors whose computed direction is rounding noise. With only two columns CGS and MGS perform the same operations, so CGS fails too. Householder QR, which never subtracts projections, gives an orthogonal Q to machine precision in both cases.

4.3 Householder reflections

Motivation. Solving least squares through $X^\top X$ squares the condition number. Orthogonal transformations preserve lengths, so reducing X to triangular form by reflections (QR) solves least squares accurately.

Proposition 4.1 (Householder reflector)

For $x, w \in \mathbb{R}^n$ with $\|x\|_2 = \|w\|_2$ and $x \neq w$, let $u = (x - w)/\|x - w\|_2$ and $H = I - 2uu^\top$. Then H is symmetric and orthogonal and $Hx = w$.

Proof. $H^\top = H$ and $H^2 = I - 4uu^\top + 4u(u^\top u)u^\top = I$. Next, $\|x - w\|^2 = 2x^\top x - 2w^\top x$ (using $x^\top x = w^\top w$), so

$$Hx = x - 2\frac{(x - w)^\top x}{\|x - w\|^2}(x - w) = x - \frac{x^\top x - w^\top x}{x^\top x - w^\top x}(x - w) = w.$$

■

Geometrically H reflects across the hyperplane orthogonal to u. Householder QR chooses H_1 with $w = \pm\|x_1\|e_1$ to zero the first column below the diagonal, then $H_2 = \text{diag}(1, \tilde{H}_2)$ acting on rows $2{:}n$ to zero the second column, and so on:

$$H_p \cdots H_2 H_1 X = \begin{pmatrix} R_1 \\ 0 \end{pmatrix}, \qquad Q = H_1 H_2 \cdots H_p.$$

This gives the *full* QR. The sign of w is chosen opposite to the first entry of x, to avoid cancellation in $x - w$: with $w = -\text{sign}(x_1)\|x\|e_1$, the first entry of $x - w$ is $x_1 + \text{sign}(x_1)\|x\|$, a sum of two numbers of the same sign. With the other choice it would be $x_1 - \text{sign}(x_1)\|x\|$, which is catastrophically small when x is nearly parallel to e_1, and u would be computed from noise.

Implementation and cost. The matrices H_j are never formed: $HX = X - 2u(u^\top X)$ costs $4np$ flops. At stage j, computing $\tilde{u}_j$ costs $3(n-j+1)$ and applying $\tilde{H}_j$ to the $(n-j+1)\times(p-j+1)$ block costs $4(n-j+1)(p-j+1)$; in total

$$\sum_{j=1}^{p} 4(n-j+1)(p-j+1) \approx 2np^2 - \tfrac{2}{3}p^3 \text{ flops.}$$

The algorithm works in place: R_1 overwrites the upper triangle, and the Householder vectors (normalized so their first entry is 1) are stored below the diagonal. To compute $Q^\top y$ or Qv one applies the reflections successively; forming Q explicitly costs another $2np^2 - \frac{2}{3}p^3$. Householder QR is backward stable and is what LAPACK (`geqrf`, or `geqp3` with column pivoting) and R's `qr` use.

Least squares without forming Q. Apply each reflection to y as soon as it is computed (cost $4(n-j+1)$ at stage j). At the end y has been overwritten by $Q^\top y = \binom{Q_1^\top y}{Q_2^\top y}$, in place and with no $n \times n$ matrix ever stored. Then

- $\hat{\beta}$ solves $R_1\beta = Q_1^\top y$ (the first p entries), by back substitution;
- $\mathrm{RSS} = \|Q_2^\top y\|^2$ is the sum of squares of the last $n-p$ entries, so $\hat{\sigma}^2 = \mathrm{RSS}/(n-p)$, or RSS/n for the maximum likelihood estimator;
- $\widehat{\mathrm{Cov}}(\hat{\beta}) = \hat{\sigma}^2 R_1^{-1} R_1^{-\top}$, from triangular solves with the R_1 already in the upper triangle.

The whole fit costs $2np^2 - \frac{2}{3}p^3 + O(np)$ flops and $O(np)$ memory.

4.4 Givens rotations

A Givens rotation $G(i,k,\theta)$ is the identity except for the entries $c, s, -s, c$ in rows and columns i, k, with $c = \cos\theta$, $s = \sin\theta$; it is orthogonal. For $y = G^\top x$,

$$y_i = cx_i - sx_k, \quad y_k = sx_i + cx_k, \quad y_j = x_j \ (j \neq i,k),$$

and the choice $c = x_i/\sqrt{x_i^2 + x_k^2}$, $s = -x_k/\sqrt{x_i^2 + x_k^2}$ makes $y_k = 0$. Applying $G^\top$ to a matrix changes only two rows ($6m$ flops). QR by Givens zeros one entry at a time, $G_t^\top \cdots G_1^\top X = \binom{R_1}{0}$, at a cost of about $3np^2 - p^3$ flops: more than Householder, but well suited to sparse or structured matrices (e.g. updating a QR when a row is added), since it touches only the entries it needs. Each rotation can be stored as a single number in the zeroed position.

4.5 Conditioning of least squares

For a nonsquare X, $\kappa(X) = \sigma_{\max}(X)/\sigma_{\min}(X) = \sigma_{\max}(X)\sigma_{\max}(X^\dagger)$, with $X^\dagger$ the Moore–Penrose pseudoinverse. The condition number of the least squares *problem* depends on the residual; it is

$$\kappa(X) + \frac{\kappa(X)^2 \tan\theta}{\eta}, \qquad \cos\theta = \frac{\|X\beta\|}{\|y\|}, \quad \eta = \frac{\|X\|\|\beta\|}{\|X\beta\|}.$$

When X is nearly collinear and the residuals are not small, the $\kappa(X)^2$ term dominates, so stable algorithms matter.

Example 4.2

$X = \begin{pmatrix} 1 & 1 \\ 10^{-3} & 0 \\ 0 & 10^{-3} \end{pmatrix}$ has full rank, but $X^\top X = \begin{pmatrix} 1+10^{-6} & 1 \\ 1 & 1+10^{-6} \end{pmatrix}$ rounds to the singular $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ with six significant digits. Forming the normal equations destroys the information, while QR applied to X itself does not.

method	flops	stability	notes
Cholesky on $X^\top X$	$np^2 + \frac{1}{3}p^3$	least stable	fastest; $X^\top X$ can be updated with streaming data
Householder QR	$2np^2 - \frac{2}{3}p^3$	stable	default in R, MATLAB, Julia
MGS	$2np^2$	stable-ish	gives Q_1 explicitly
SVD	$4n^2p + 8np^2 + 9p^3$	most stable	also handles rank deficiency

There is no universal gold standard: Cholesky is twice as fast for $n \gg p$ and needs only $X^\top X$ (which fits in memory even when X does not), while QR is more accurate.

4.6 Underdetermined least squares

If $\operatorname{rank} X < p$ (in particular $n < p$), the normal equations have infinitely many solutions, and an extra criterion is needed.

- **Minimum ℓ_2 norm**: minimize $\|\beta\|_2$ subject to $X^\top X\beta = X^\top y$. The solution is $\hat\beta = X^\dagger y$, which for full row rank X ($n < p$) is $X^\top(XX^\top)^{-1}y$. It is also the limit of ridge regression: $\hat\beta = \lim_{\lambda\downarrow 0}(X^\top X + \lambda I)^{-1}X^\top y$.
- **Minimum ℓ_0 "norm"**: minimize $\|\beta\|_0 = \#\{j : \beta_j \neq 0\}$, the sparsest solution. This is best subset selection, a combinatorial problem.
- **Minimum ℓ_1 norm**: a convex relaxation (basis pursuit, related to the lasso), solvable by linear programming, which under suitable conditions recovers the sparsest solution.